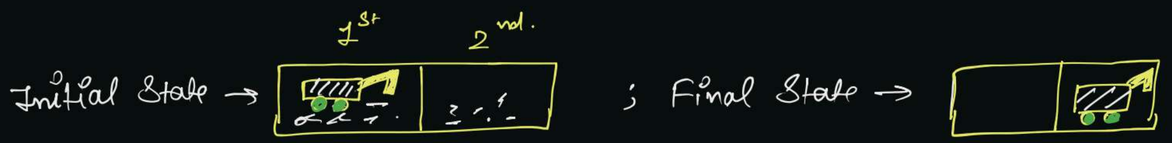
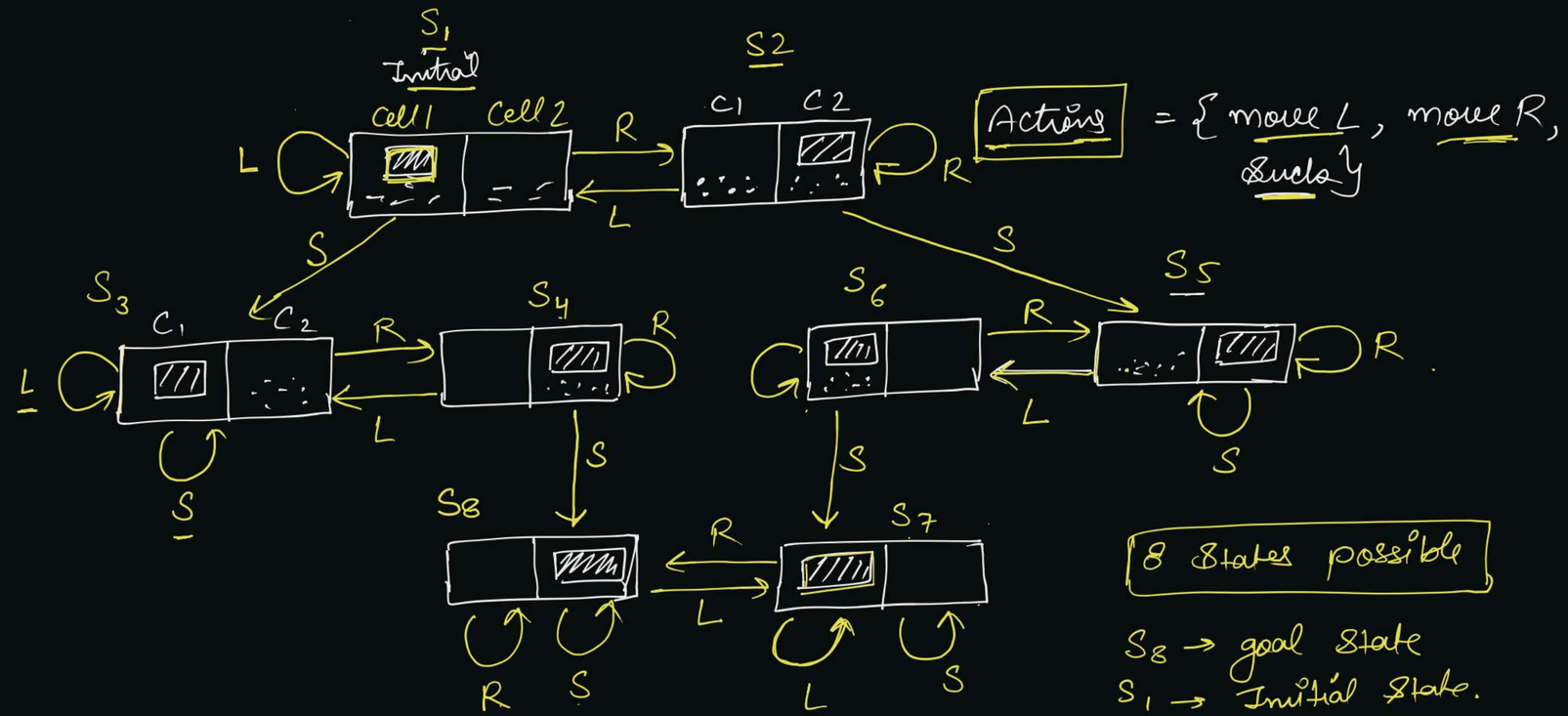
HW

Eg. Consider a two-cell vacuum world problem,



Cells contain trash, agent can move left, right or Suck the trash. Create a state-space graph.

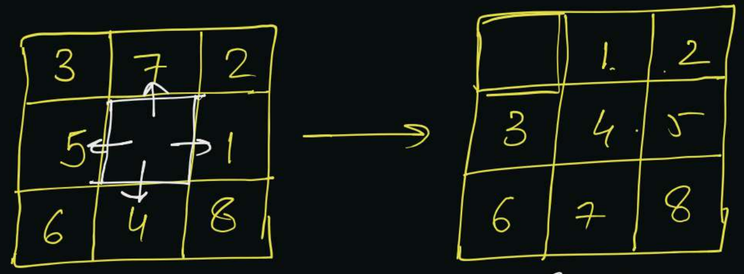
Sol.



H.W.

Consider the 8-puzzle problem.

States = ?
actions = ?



Starting State.

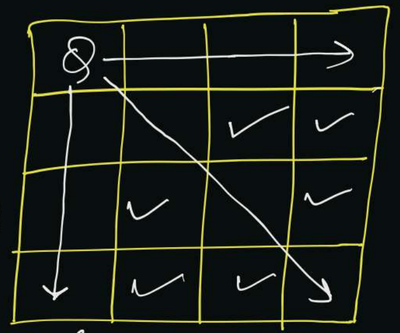
Goal State.

H.W.

Consider n-Queens problem.

↳ Rule : no queens should intersect paths.

↳ Goal : Place n-queens (4) on n×n geid that follows the rule.



n = 4.
4 → Queens.

3. Uninformed Search

Tuesday, 10 September 2024 9:27 PM

Measuring problem-solving performance :-

1. **Completeness** :- Is the algorithm guaranteed to find a solution, if there is any. And should correctly report failure when there is not.

2. **Cost optimality** :- Are we able to find a solution with the lowest path cost among all solutions.

3. **Time complexity** :- How long does it takes to find a solution. (Worst case)

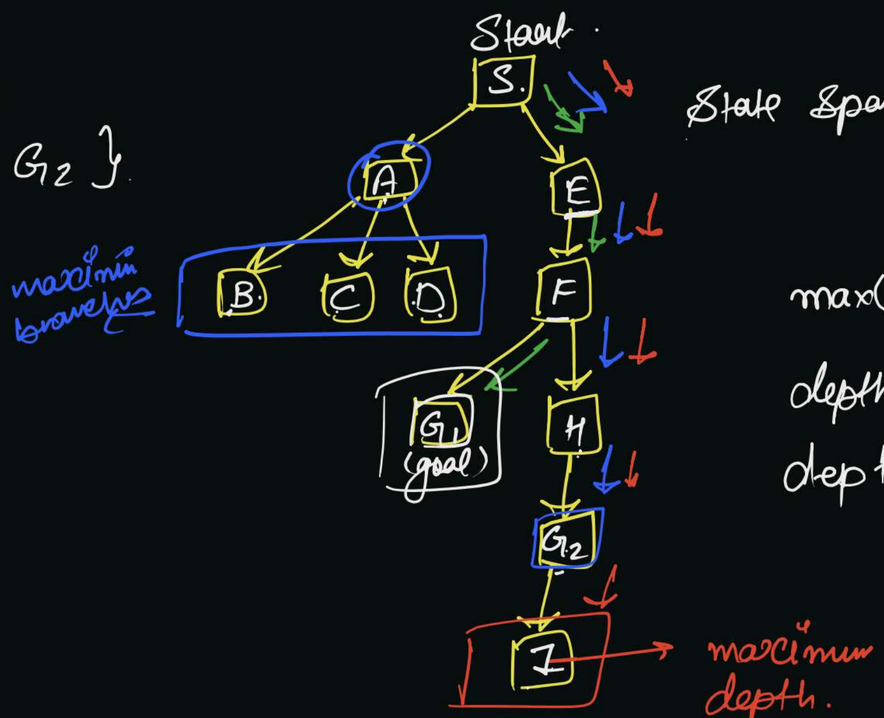
4. **Space complexity** :- How much memory does it takes to perform the search.

Parameters related to Search :-

1. **Branching Factor (b)** :- Maximum number of successors of any state.
2. **Shallowest goal's depth (d)** → Least possible depth of a goal state from start (initial) state.
3. **Maximum depth of State Space (m)** → Max. possible depth among all possible states (State Space set).

Initial State = { S }
goal State = { G_1, G_2 }

b = 3
d = 3
m = 5



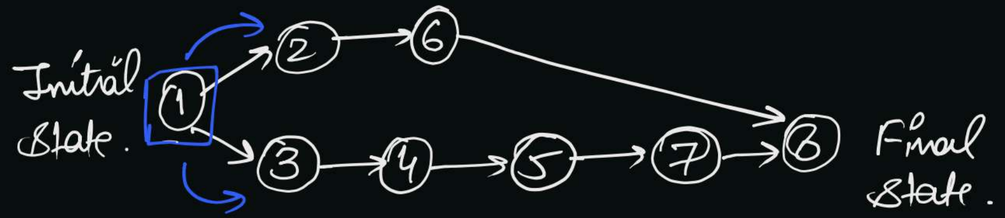
State Space = { S, A, B, C, D, E, F, G_1, G_2, H, I }
Start State.
goal State.

max(b) = 3 = b.

depth (G_1) = 3
depth (G_2) = 4
} min = 3 = d.

Uninformed Search :-

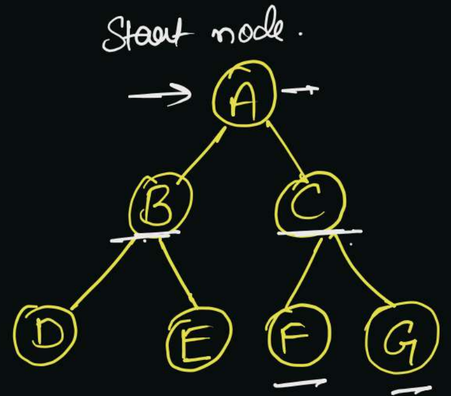
↳ These algorithms has no clue about how close a state is to the goal.

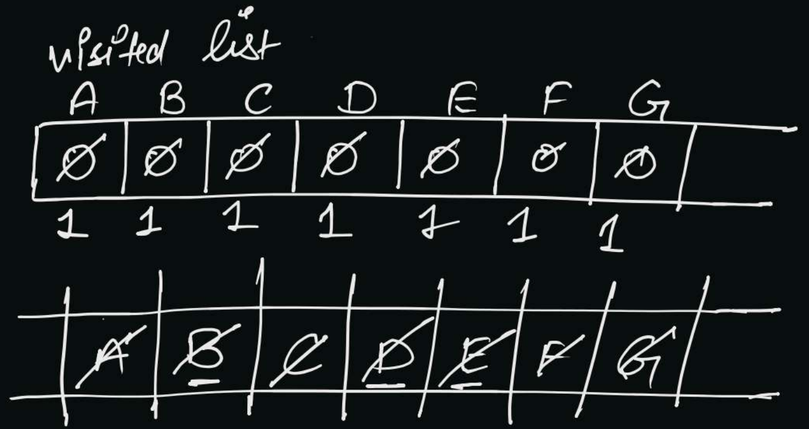


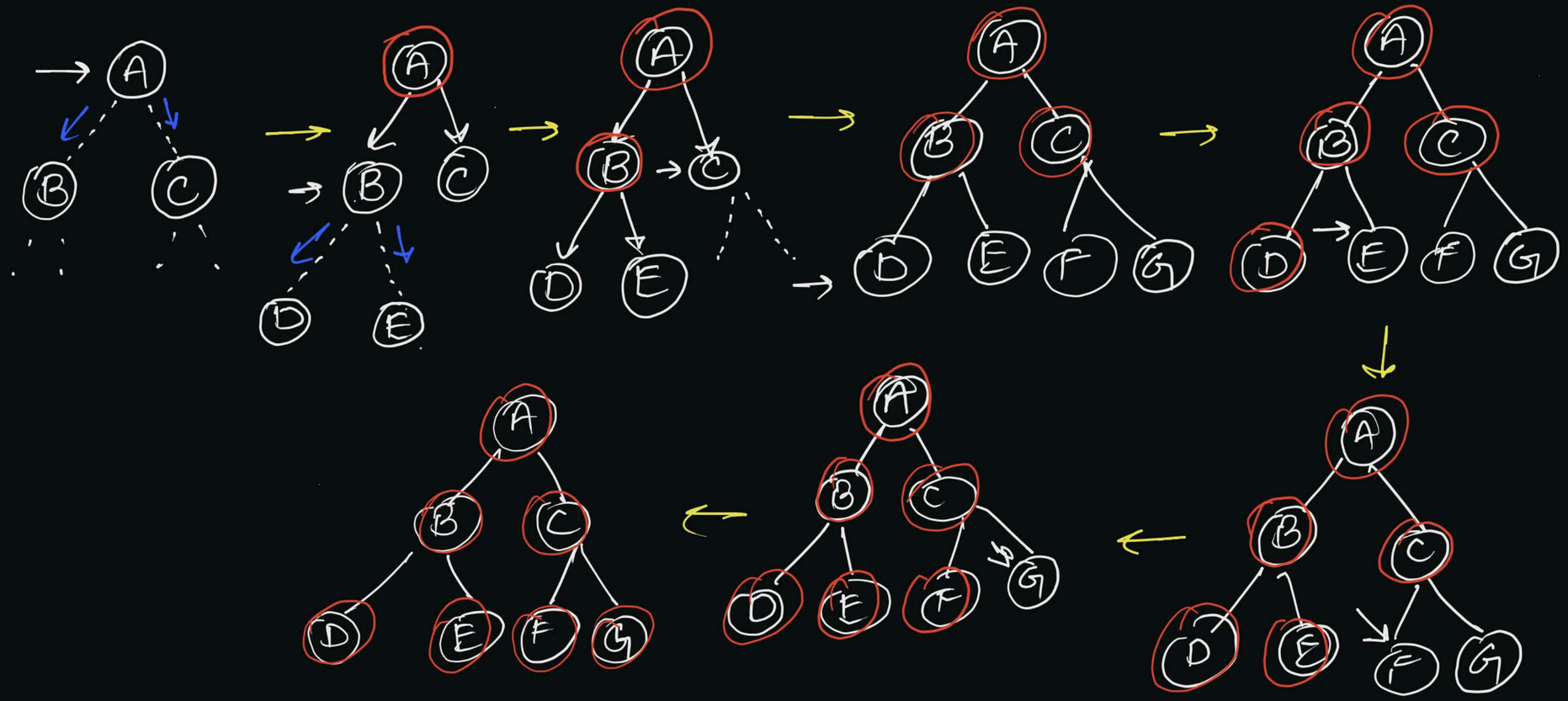
↳ Also known as blind search algorithms.

↳ Algorithms → BFS, DFS, Uniform-cost, depth-limited depth first iterative deepening, bidirectional.

1. **BFS** :- "Expand shallowest unexpanded node."






A
B
C
D
E
F
G

Pseudo - algorithm :-

take q as an empty queue.
q.enqueue(S)
marks s as visited.
→ while (q is not empty) ⟷ looping until the queue becomes empty.
{
  t = q.dequeue() ←
  for all neighbours t' of t → exploring all the neighbours of t.
  {
    ↳ if (t' ≡ g) → return t'. → terminate the search if the goal is found.
    ↳ if (t' is not visited)
    {
      q.enqueue(t').
      marks t' as visited.
    }
  }
}

termination conditions

(q) → queue
(s) → starting node.
(g) → goal state.

goal = {F}

S	B	C	D	E

Iteration 1 :
t = S A
t' = {B, C}

Iteration 2 :
t = B
t' = {D, E}